

This is the **Master Architecture Document** for the **Velites** repository.

This document serves as the single source of truth for the development of your Quantamental Research Agent. It defines the boundaries, inputs, outputs, and logic for every module.

## Repository Name: Velites

**Mission:** To act as the autonomous forward scout for the "Homeguard" execution system. Velites identifies "Information Asymmetry" in the AI/Semiconductor supply chain by analyzing technical literature and validating it against market news before price discovery occurs.

---

## System Architecture Overview

---

### Module 1: The Scout (Data Ingestion)

Directory: /modules/scout

Role: The "Senses". Responsible for fetching raw, unstructured data from external sources. It performs no analysis, only normalization.

#### 1.1 The Librarian (Technical Discovery)

- **Goal:** Detect pre-product innovation signals.
- **Data Sources:**
  - **ArXiv API:** Categories cs.AI, cs.LG (Machine Learning), cs.AR (Hardware), cs.CV (Computer Vision).
  - **Google Patents / USPTO:** RSS feeds for specific assignee names (e.g., "TSMC", "Advanced Micro Devices").
- **Logic:**
  - Runs on a schedule (e.g., 06:00 EST).
  - Filters results by date (Last 24h).
  - Discards papers with generic keywords (e.g., "Survey of...", "Review of...").
- **Output:** PaperObject (Title, Abstract, Authors, URL, PublishedDate).

#### 1.2 The News Aggregator (Macro Validation)

- **Goal:** Fetch business context to validate technical signals (The "Reality Check").
- **Data Sources:**
  - **Tiingo / NewsData.io:** Semantic search.
  - **Niche RSS:** *AnandTech*, *Tom's Hardware*, *Semianalysis*, *DigiTimes*.
- **Logic:**
  - Fetches news based on dynamic watchlists generated by Module 2.
  - Prioritizes "Supply Chain" keywords (e.g., "Capacity", "Yield", "Shortage", "Delay").
- **Output:** NewsObject (Headline, Summary, Source, Timestamp, Content).

## 1.3 The Market Feeder (Price Reality)

- **Goal:** Provide current market state to prevent trading into illiquidity.
  - **Data Sources:** yfinance (Dev), Alpaca Data API (Prod).
  - **Logic:**
    - Fetches **OHLCV** (Open, High, Low, Close, Volume).
    - **Crucial:** Fetches **Bid/Ask Spread** snapshot for OTC tickers.
  - **Output:** MarketState (Ticker, Price, Volume\_30d\_Avg, Spread\_Pct, Liquidity\_Status).
- 

## Module 2: The Mapper (Knowledge Graph)

Directory: /modules/mapper

Role: The "Brain". The core IP that translates text into tradeable entities.

Source File: data/knowledge\_graph\_v1\_2.json

### 2.1 The Entity Resolver

- **Goal:** Map unstructured text to specific graph nodes.
- **Input:** Raw text (e.g., "HBM3e yields are low").
- **Logic:**
  - Loads product\_map and semiconductor\_software from JSON.
  - Uses Fuzzy Matching (Levenshtein distance) to handle typos.
  - Resolves synonyms (e.g., "Blackwell"  $\rightarrow$  NVDA, "GAAFET"  $\rightarrow$  TSM).
- **Output:** List of EntityNode (Ticker, Role, Confidence).

### 2.2 The Supply Chain Navigator

- **Goal:** Identify second-order effects (The "Pick & Shovel" trade).
- **Input:** Primary Ticker (e.g., NVDA).
- **Logic:**
  - Traverses the graph in knowledge\_graph\_v1\_2.json.
  - **Upstream:** "Who supplies NVDA?"  $\rightarrow$  TSM, SKHIY.
  - **Downstream:** "Who buys from NVDA?"  $\rightarrow$  MSFT, META.
  - **Lateral:** "Who makes the tools?"  $\rightarrow$  SNPS (EDA), CAMT (Inspection).
- **Output:** DependencyMap (Tier 1 Suppliers, Critical Bottlenecks).

### 2.3 The Ticker Normalizer

- **Goal:** Ensure tradeability on US exchanges (Alpaca/IBKR).
- **Input:** International Ticker (e.g., 005930.KS - Samsung).
- **Logic:**
  - Look up ticker\_normalization in JSON.
  - Return the us\_adr (e.g., SSNLF).

- **Flag Risk:** If us\_liquidity is "low", flag as LIMIT\_ORDER\_ONLY.
  - **Output:** TradeableTicker (Symbol, Venue, RiskFlag).
- 

## Module 3: The Analyst (Signal Generation)

Directory: /modules/analyst

Role: The "Processor". Interprets data significance using AI.

### 3.1 The Innovation Agent (LLM)

- **Goal:** Rate the significance of technical findings.
- **Tech:** OpenAI GPT-4o / Anthropic Claude 3.5 Sonnet.
- **Input:** Abstract from Module 1.1 + Context from Module 2.1.
- **Prompt:** *"Does this paper describe a material breakthrough for [Ticker]? Rate -1.0 to +1.0."*
- **Output:** InnovationScore (Float), Reasoning (String).

### 3.2 The Sentiment Agent (Guardrail)

- **Goal:** Prevent fighting the macro trend (The Veto).
- **Tech:** FinBERT (Hugging Face).
- **Input:** Headlines from Module 1.2.
- **Logic:**
  - Calculate weighted average sentiment of last 24h news.
  - **VETO Condition:** If Sentiment < -0.6 (Deep Negative), block Long signals.
- **Output:** SentimentScore (Float), HypeVolume (Z-Score).

### 3.3 The Confluence Engine

- **Goal:** The final decision matrix.
  - **Logic:**
    - Combine InnovationScore + SentimentScore.
    - Apply **Time-Lag Logic:** If Source is ArXiv (Leading) and News is Quiet (Lagging)  $\rightarrow$  **High Confidence**.
    - Apply **Hype Filter:** If News Volume >  $3\sigma$  (Sigma)  $\rightarrow$  **Hold/Wait** (Too crowded).
  - **Output:** AlphaSignal (Action: BUY/WAIT/IGNORE, Target, Confidence).
- 

## Module 4: The Courier (Interface)

Directory: /modules/courier

Role: The "Messenger". Formats and delivers the order to the Homeguard Repo.

Note: This module does NOT execute trades. It dispatches instructions.

## 4.1 The Payload Formatter

- **Goal:** Create a standardized JSON contract for Homeguard.
- **Output Format:**

```
JSON
{
  "source": "Velites_v1",
  "timestamp": "2026-01-16T12:00:00Z",
  "type": "QUANTAMENTAL_LONG",
  "ticker": "CAMT",
  "venue": "NASDAQ",
  "order_type": "LIMIT",
  "limit_price": 78.50,
  "validity": "24h",
  "reasoning": "ArXiv paper #2401.1234 indicates high demand for 3D metrology. News sentiment neutral.",
  "risk_flags": ["SMALL_CAP", "EARNINGS_APPROACHING"]
}
```

## 4.2 The Liquidity Guard (Final Check)

- **Goal:** Hard rejection of un-tradeable assets.
- **Logic:**
  - Checks Module 1.3 Market Data.
  - **Rule:** If Spread > 2%, convert signal to NO\_GO.
  - **Rule:** If Volume < \$500k, convert signal to NO\_GO.

---

# Module 5: The Scribe (Logging)

Directory: /modules/scribe

Role: The "Black Box".

## 5.1 The Journal

- **Goal:** Persistent record for backtesting.
- **Storage:** SQLite (local) or PostgreSQL.
- **Schema:** (Date, SignalID, Ticker, Input\_Paper, LLM\_Reasoning, Market\_Price\_At\_Signal, Outcome\_7d).

---

# Recommended Tech Stack

| Component   | Library/Tool                               |
|-------------|--------------------------------------------|
| Language    | Python 3.11+                               |
| Graph DB    | NetworkX (In-memory) + JSON                |
| LLM         | OpenAI API (openai) / LangChain            |
| NLP         | Transformers (transformers, torch)         |
| Scraping    | feedparser, beautifulsoup4, requests       |
| Market Data | yfinance                                   |
| Environment | Docker (separate container from Homeguard) |

## Implementation Priority

1. **Mapper:** Build knowledge\_graph.py to parse your JSON.
2. **Scout:** Build the ArXiv fetcher.
3. **Analyst:** Connect the LLM.
4. **Courier:** Define the JSON payload.